

Program No:12

Aim

To implement a concurrent Time Server using UDP in Python, where multiple clients can send requests and receive the current server time simultaneously.

Algorithm

Server

1. Start the program.
2. Import socket, threading, and datetime.
3. Create a UDP socket.
4. Bind the socket to IP address 127.0.0.1 and port 5000.
5. Wait for a client request using `recvfrom()`.
6. When a request is received, create a **new thread**.
7. In the thread, get the current system time.
8. Send the time to the client using `sendto()`.
9. Continue waiting for other client requests.
10. Stop the server when the program is terminated.

Client

1. Start the program.
2. Create a UDP socket.
3. Set the server address as 127.0.0.1 and port 5000.
4. Send a "TIME" request to the server.
5. Receive the server time using `recvfrom()`.
6. Display the received time.
7. Close the socket.
8. Stop.

UDP Concurrent Time Server

```
import socket
import threading
from datetime import datetime

def send_time(data, address):
    time = datetime.now().strftime("%H:%M:%S")
    server.sendto(time.encode(), address)

server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

server.bind(("127.0.0.1", 5000))

print("Server is running...")

while True:
    data, address = server.recvfrom(1024)

    print("Request received from:", address)
```

```
# Create a new thread for each client
t = threading.Thread(target=send_time, args=(data, address))
t.start()
```

UDP Client

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

server_address = ("127.0.0.1", 5000)

client.sendto(b"TIME", server_address)

data, address = client.recvfrom(1024)

print("Server Time:", data.decode())

client.close()
```

Result

A concurrent UDP Time Server was successfully implemented in Python. The server receives requests from clients using UDP and sends the current system time to each client using separate threads.